

Portfolio personale dinamico

Proposta progettuale

1. Descrizione generale

Il progetto consiste nella realizzazione di un portfolio personale dinamico, sviluppato come applicazione web completa composta da frontend, backend e database. Il sito presenta il profilo, le competenze, il percorso formativo e i progetti realizzati dal proprietario del portfolio, ed è affiancato da un'area amministrativa protetta che permette di gestire i contenuti senza intervenire manualmente sul codice o sul database.

I visitatori potranno consultare i progetti, cercarli e filtrarli per categoria, visualizzare una pagina di approfondimento per ciascun progetto, scaricare il curriculum e inviare un messaggio tramite un modulo di contatto. I contenuti relativi a progetti e competenze non saranno inseriti direttamente nel codice HTML, ma recuperati dinamicamente da un database MySQL attraverso un server Node.js.

2. Tecnologie utilizzate

- HTML5 per la struttura semantica delle pagine
- CSS3 per layout, stile, tema chiaro/scuro e responsive design
- JavaScript vanilla per DOM, eventi, validazione form e richieste `fetch`
- Node.js ed Express per il routing, i middleware e le API REST
- MySQL per la persistenza dei dati, con `mysql2/promise` e prepared statement
- JWT per l'autenticazione dell'area amministrativa
- Cookie `HttpOnly` per la conservazione sicura del token di sessione

Non vengono utilizzati framework frontend pesanti come Bootstrap, Tailwind, React o Angular.

3. Pagine previste

Area pubblica

- `index.html` — **Home**: nome e ruolo professionale, breve introduzione, pulsanti per progetti e curriculum, progetti in evidenza caricati dinamicamente, descrizione del metodo di lavoro, collegamenti a email, GitHub e LinkedIn
- `about.html` — **Profilo**: biografia, percorso formativo, competenze tecniche raggruppate per categoria e caricate dal database, timeline di esperienze, collegamento al curriculum
- `projects.html` — **Progetti**: elenco completo dei progetti con ricerca per titolo/descrizione/tecnologia, filtro per categoria, ordinamento alfabetico o cronologico, conteggio dei risultati, messaggio in caso di nessun risultato, card generate dinamicamente da JavaScript
- `project.html` — **Dettaglio progetto**: identificato tramite parametro URL `?id=`, mostra titolo, immagine, descrizione completa, categoria, tecnologie, anno, problema affrontato, soluzione adottata, collegamento al repository ed eventuale demo
- `contact.html` — **Contatti**: email, collegamenti social, modulo con campi nome/email/oggetto/messaggio, validazione e messaggi di conferma o errore, invio asincrono senza ricaricare la pagina
- `404.html` — **Pagina di errore**: messaggio, breve spiegazione, collegamento alla home o alla pagina progetti

Area amministrativa

- `admin-login.html` — form di accesso per l'amministratore
- `admin-dashboard.html` — elenco dei progetti con possibilità di ricerca, modifica ed eliminazione
- `admin-project-form.html` — creazione e modifica di un progetto
- `admin-skills.html` — creazione, modifica ed eliminazione delle competenze
- `admin-messages.html` — lettura ed eliminazione dei messaggi ricevuti dal form di contatto

Tutte le pagine admin, ad eccezione del login, saranno protette da un middleware che verifica la validità del token JWT contenuto nel cookie di sessione.

4. Funzionalità e interazioni con l'utente

Navigazione responsive

Il menu sarà orizzontale su desktop e diventerà un menu compatto su smartphone.

Caricamento dinamico dei progetti

JavaScript richiederà al server i progetti, riceverà una risposta JSON e creerà dinamicamente le card nel DOM.

Ricerca, filtri e ordinamento

L'utente potrà combinare ricerca testuale, categoria e ordinamento. L'elenco verrà aggiornato senza ricaricare la pagina.

Dettaglio dinamico

JavaScript leggerà il parametro `id` presente nell'URL e mostrerà il progetto corrispondente oppure un messaggio di risorsa non trovata.

Competenze dinamiche

Le competenze saranno richieste al server, raggruppate e inserite dinamicamente nella pagina Profilo.

Tema grafico

Il visitatore potrà scegliere tra tema chiaro e scuro. La preferenza sarà memorizzata nel browser tramite `localStorage`.

Gestione contenuti dall'area admin

L'amministratore, una volta autenticato, potrà creare, modificare ed eliminare progetti e competenze, oltre a consultare ed eliminare i messaggi ricevuti dal form contatti, direttamente dal browser.

Feedback all'utente

Saranno presenti indicatori di caricamento, messaggi di errore e conferma, stato senza risultati, notifiche temporanee e focus visibile.

5. Flusso di autenticazione amministratore

1. L'admin invia email e password dal form di login.
2. Il backend recupera l'utente dalla tabella `admin_users`.
3. La password viene verificata ricalcolando l'hash PBKDF2 con il salt salvato.
4. Il backend firma un JWT con una chiave segreta, con scadenza configurabile.
5. Il token viene salvato in un cookie `HttpOnly`, `SameSite=Lax` (`Secure` in produzione).
6. Le richieste successive alle API admin vengono validate leggendo e verificando il cookie.
7. Il logout invalida il cookie lato server.

6. Informazioni da memorizzare nel database

Il database MySQL sarà denominato `portfolio_db`.

Tabella	Contenuto e utilizzo
<code>projects</code>	Elenco dei progetti e informazioni complete utilizzate nella pagina di dettaglio
<code>skills</code>	Competenze organizzate in gruppi (Frontend, Backend, Database, Strumenti)
<code>messages</code>	Messaggi inviati dai visitatori tramite il modulo della pagina Contatti
<code>admin_users</code>	Account amministratore: email, hash della password (PBKDF2) e salt

Per l'accesso al database verrà utilizzato un connection pool MySQL. Le query contenenti dati inseriti dagli utenti useranno prepared statement con placeholder, limitando il rischio di SQL injection.

7. Operazioni lato server

API pubbliche

```
GET /api/health
GET /api/projects
GET /api/projects/:id
```

```
GET /api/skills
POST /api/messages
```

API amministrative

```
POST /api/admin/login
GET /api/admin/me
POST /api/admin/logout

GET /api/admin/projects
GET /api/admin/projects/:id
POST /api/admin/projects
PUT /api/admin/projects/:id
DELETE /api/admin/projects/:id

GET /api/admin/skills
GET /api/admin/skills/:id
POST /api/admin/skills
PUT /api/admin/skills/:id
DELETE /api/admin/skills/:id

GET /api/admin/messages
GET /api/admin/messages/:id
DELETE /api/admin/messages/:id
```

Tutte le rotte admin, tranne il login, sono protette da un middleware di autenticazione che verifica firma e scadenza del token JWT.

8. Sicurezza

- Prepared statement MySQL su tutte le query con input utente
- JWT firmato lato server, mai esposto in query string
- Cookie `HttpOnly` + `SameSite=Lax` (`Secure` in produzione)
- Password amministratore con salt casuale e hash PBKDF2 (nessuna password in chiaro nel database)
- Header di sicurezza HTTP minimi: `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`
- Validazione input sia lato client sia lato server
- Protezione server-side delle rotte e delle pagine admin

9. Scelte stilistiche

Il design sarà minimale, elegante e professionale. L'interfaccia utilizzerà ampi spazi vuoti, gerarchia tipografica chiara, bordi sottili, ombre leggere, angoli arrotondati, colori caldi e naturali, animazioni discrete ed elementi interattivi chiaramente riconoscibili.

PaLETTE principale chiara

Sfondo #F7F3EC	Superficie #FFFDF8	Superficie sec. #F0E9DF	Testo #26211D	Accento #C96442	Accento scuro #9F472D	Successo #2F6F4E	Errore #A53A31
-------------------	-----------------------	----------------------------	------------------	--------------------	--------------------------	---------------------	-------------------

Tema scuro

Sfondo #1F1B18	Superficie #29231F	Accento #E38361	Successo #76BD94	Errore #EF8C83
-------------------	-----------------------	--------------------	---------------------	-------------------

Font

I font utilizzati saranno:

- `Arial, Helvetica, sans-serif` — font principale per corpo del testo, menu e interfaccia
- `Georgia, "Times New Roman", serif` — font secondario per titoli editoriali e alcuni elementi decorativi

La gerarchia visiva sarà ottenuta variando dimensione, peso, spaziatura, colore e posizione.

10. Responsive design

Il sito sarà progettato secondo un approccio responsive utilizzando media query, Flexbox e CSS Grid.

Desktop

- layout a più colonne
- menu orizzontale
- griglie di progetti
- contenuti affiancati

Smartphone

- una sola colonna
- menu compatto
- pulsanti a larghezza maggiore
- card impilate verticalmente
- spaziature ridotte
- testi ridimensionati con unità relative

11. Conclusione

Il progetto consentirà di applicare in modo integrato gli argomenti affrontati durante il corso: HTML semantico, CSS responsive, progettazione dell'interfaccia, manipolazione del DOM, gestione degli eventi, comunicazione asincrona, Node.js, Express e database MySQL. L'area amministrativa protetta da autenticazione JWT aggiunge inoltre un livello di gestione completa dei contenuti, dimostrando competenze di sicurezza applicativa lato backend. Il risultato finale sarà un portfolio dinamico, accessibile, responsive, sicuro e collegato a un database operativo.